What is CPU Pipelining?

Breaking the Cycle

In a standard Von Neumann CPU, the cycle is **Fetch → Decode → Execute**.

In a **Pipelined CPU**, we split these into separate hardware stations that work simultaneously.

- While Instruction 1 is **Executing**...
- Instruction 2 is being **Decoded**...
- Instruction 3 is being **Fetched**.

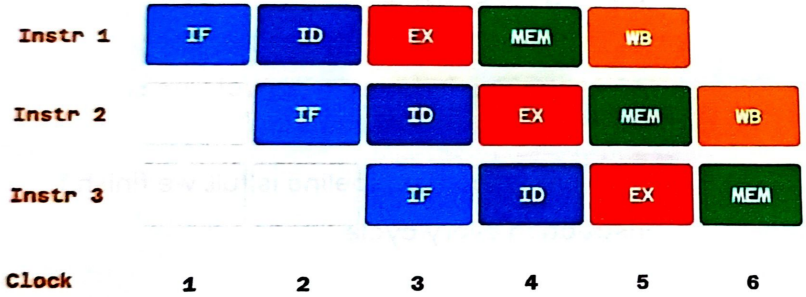
> **Pipelining**
> An implementation technique where multiple instructions are overlapped in execution. The goal is to increase the number of instructions completed per second (Throughput).



Assembly Line Manufacturing: Everyone keeps working.

The 5-Stage RISC Pipeline

Goal: One instruction completes every clock cycle (Throughput = 1).



IF:
Instruction Fetch |
ID:
Decode & Register Read |
EX:
Execute/ALU |
MEM:
Memory Access |
WB:
Write Back

Latency vs. Throughput

Latency

Time to finish **one** single task.

Example: It takes 5 cycles (IF, ID, EX, MEM, WB) to finish one instruction fully.

Pipelining does NOT improve Latency.

Throughput

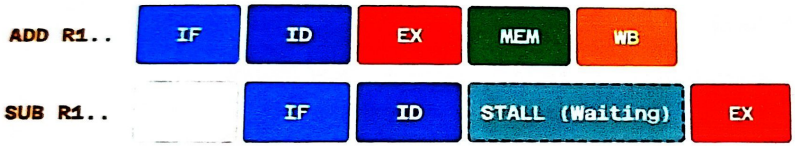
Amount of work done per unit of time.

Example: Once the pipeline is full, we finish 1 instruction every cycle.

Pipelining drastically improves Throughput.

When the Pipeline Breaks

In reality, one instruction might block the next. This creates a "Stall".



Pipeline

A situation where the next instruction cannot execute in the next clock cycle.

Bubble / Stall

A delay (NOP - No Operation) injected into the pipeline to wait for the hazard to clear.

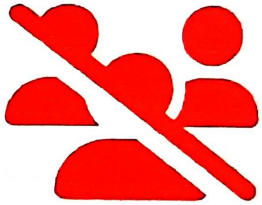
1. Structural Hazards

Resource Conflict

Problem: Two instructions try to use the same piece of hardware at the same time.

Example: If Instr 1 is writing data to RAM (MEM stage), and Instr 4 is trying to fetch code from RAM (IF stage), they clash if there is only one memory port.

Solution: Separate L1 Caches (Instruction Cache vs. Data Cache) - Harvard Architecture.



Analogy: Two people needing the dryer at once.

2. Data Hazards (RAW)



Read-After-Write (RAW)

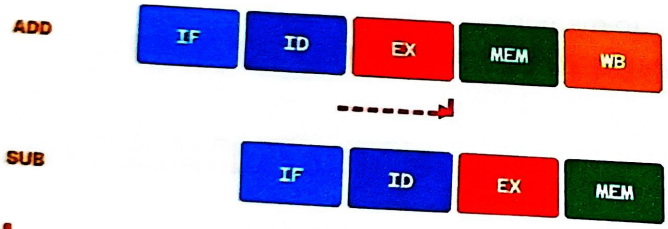
Instruction B needs the result of Instruction A **before** A has written it to memory.

```
ADD R1, R2, R3   ; Writes R1 at stage 5 (WB)
SUB R4, R1, R5   ; Needs R1 at stage 2 (ID)
```

The SUB instruction has to wait (Stall) until ADD finishes writing. This wastes cycles.

Solution: Data Forwarding

Instead of stalling, we wire the output of the ALU directly to the input of the next ALU cycle.



Forwarding (Bypassing):
A hardware technique where the result is grabbed from the pipeline register (end of EX stage) and fed back immediately, skipping the write-to-memory step.

3. Control Hazards

The "Branch" Problem

Pipelining assumes we know what to fetch next (PC + 1).

But what if we hit an IF statement (Branch)?

```
BEQ target    ; If Equal, goto Target
MOV R1, #5    ; ... pipeline fetches this
target:       ; ... but maybe we go here?
```

The CPU doesn't know the result of the IF until the Execute stage. By then, it has already fetched the wrong instructions.



Fork in the road.

Solution: Branch Prediction

Guessing

The CPU guesses: "I think this IF will be TRUE".



Speculative Exec

It keeps fetching instructions from the guessed path.



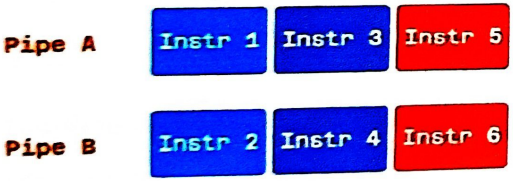
Flush

If guessed wrong, it flushes the pipeline (throws away work) and starts over. This is a penalty.

Static Prediction:
Always assume branch is NOT taken.
Dynamic Prediction:
Use history tables (AI-lite) to predict based on past behavior. Modern CPUs are >99% accurate.

Superscalar: Multiple Lanes

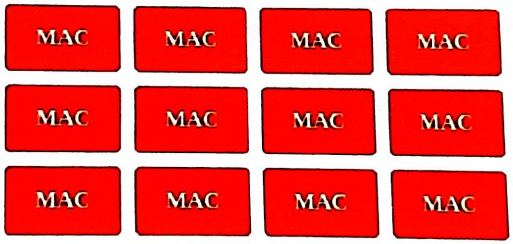
Why have 1 pipeline when you can have 2 (or 4)?



Superscalar Architecture
A CPU that implements instruction-level parallelism within a single processor. It allows the CPU to execute more than one instruction per clock cycle (IPC > 1).

Why AI Loves Pipelines

AI is mostly **Matrix Multiplication**. This is deterministic (no branches).



Systolic Array (TPU Architecture):

Instead of Fetch-Decode-Execute for every number, data flows through a grid of ALUs like heartbeats (systole). Data flows in from the left, weights stay stationary, and results flow down.

Result:

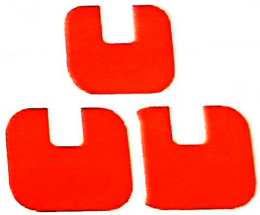
100% Pipeline Utilization. No Stalls.

Optimization: Batching

Filling the Pipe

If you process 1 image at a time, you pay the "Latency Cost" (filling the pipeline) every time.

If you process a **Batch** of 64 images, you fill the pipeline once and then stream data at maximum throughput.



> **Amortization:**
> Spreading the initial cost (filling the pipeline) over a large number of items (the batch), making the average cost per item very low.

CPU vs. GPU Pipelining

Feature	CPU (Intel Core i9)	GPU (NVIDIA H100)
Pipeline Depth	Deep (15-20 stages)	Simple
Goal	Low Latency (Fast single task)	High Throughput (Massive parallel tasks)
Branch Prediction	Extremely Complex (Smart)	Simple (Dumb)
Handling Hazards	Hardware re-ordering	**Thread Switching** (Hide Latency)

Latency Hiding (GPU):
When a GPU thread hits a hazard (needs memory), it immediately switches to another thread that is ready to run, ensuring the hardware never goes idle.

Calculate the Speedup

Scenario

- **Task:** Process 100 Instructions.
- **Non-Pipelined:** 1 instruction takes 5 cycles.
- **Pipelined:** 1 instruction finishes every 1 cycle (after fill).

Calculation

Old Time: 100 * 5 = 500 Cycles

New Time: 100 + 4 (fill) = 104 Cycles

Speedup: 500 / 104 ≈ **4.8x Faster**

(Ideal speedup is 5x, but overhead reduces it).

Amdahl's Law:
The theoretical speedup is always limited by the serial part of the task (the fill time).